Architecture Decision Records (ADR) — Стандартотворець

Короткі записи нетривіальних рішень, щоб майбутній агент (або майбутній ти) не переоткривав колесо. Формат кожного запису: **Контекст / Рішення / Альтернативи / Наслідки**. Найновіші — зверху. Нумерація наскрізна (ADR-NNNN).

> Додавай запис щоразу, коли приймаєш рішення, яке хтось згодом може поставити під сумнів ("а чому саме так?"). Коміт ADR — окремий від коду (`docs: ADR-NNNN …`).

ADR-0008 — Шаблон задач — це прапорець на стандарті, дати перераховуються за етапами

- **Дата:** 2026-08-03
- **Контекст:** У кожної РГ стандарти проходять однаковий шлях — ТЗ, проєкт, відгуки, технічна перевірка, фінальний пакет. Набір задач під цей шлях у СТД-100-200-001 склався природно, і його хотілося розмножити на решту. Пряме копіювання дає сміттєві дедлайни: у джерела «Субперевірка ТЗ» стоїть на 14 травня, і в цільовому стандарті вона опинилась би на тій самій даті, хоча його ТЗ здається у вересні.
- **Рішення:** Шаблон — не окрема сутність, а `Standard.isTaskTemplate Boolean @default(false)`. Будь-який стандарт можна позначити шаблоном, `standard.templates` віддає позначені, `task.copyFromTemplate` клонує задачі з підзадачами в цільовий. **Дати перераховуються, а не копіюються:** для кожної вихідної задачі шукається найближчий етап поетапного плану джерела, запам'ятовується зсув у днях від нього, і в цільовому стандарті дата = відповідний етап цілі + той самий зсув. Задача без дати лишається без дати; цільовий етап без дати — задача теж лишається порожньою. Виконавці скидаються (`resetAssignees: true`), бо склад РГ різний. Режими `append` / `replace`; при `replace` існуючі задачі видаляються в тій самій транзакції.
- **Альтернативи:**
  - **Окрема модель `TaskTemplate` + `TaskTemplateItem`** — друга паралельна ієрархія, яку треба окремо редагувати, і яка одразу розходиться з живими задачами. Прапорець дає редагування шаблону тим самим UI, що й звичайних задач.
  - **Копіювати дати як є** — див. контекст, дедлайни з минулого.
  - **Зсув від дати створення стандарту** — не тримається за зміст: «Субперевірка ТЗ» прив'язана до етапу ТЗ, а не до того, коли стандарт завели.
- **Наслідки:** Шаблон — звичайний стандарт, він видно в списках і рахується в статистиці; якщо це заважатиме, знадобиться окремий фільтр. Перерахунок дат залежить від того, що в цільового стандарту заповнений поетапний план — без дат етапів задачі приїдуть без дедлайнів. Зміна

набору етапів (`StandardStage`) вимагає оновити таблицю відповідності і в `task.copyFromTemplate` .

ADR-0007 — Підзадача — плоский чекліст, а не вкладена задача

- **Дата:** 2026-07-14
- **Контекст:** Задачі в РГ природно розпадаються на кроки («Засідання РГ — початок робіт», «Субперевірка ТЗ», «Погодження в замовника»). Напрошувалось зробити `Task.parentId` і дозволити довільну вкладеність.
- **Рішення:** Окрема модель `TaskChecklistItem` з каскадом від `Task` і рівно одним рівнем вкладеності. Поля дзеркалять задачу (`title` , `description` , `isDone` , `dueDate` , `assigneeId` , `order`), але це **не** `Task` : немає статусу-машини, пріоритету, власного аудиту й власної сторінки. Порядок — явне поле `order` , перетягуванням.
- **Альтернативи:**
  - **`Task.parentId` + самозв'язок** — одразу тягне за собою питання, що таке «прострочена» задача з виконаними дітьми, як рахувати KPI на дашборді, чи успадковується виконавець, що робити з трирівневою вкладеністю. Ціна — переписування всіх запитів `task.list` , дашборда й звітів.
  - **Чекліст як Markdown-текст у `Task.description`** — не дає ні виконавця, ні дедлайна, ні порядку, ні відмітки виконання.
- **Наслідки:** Третій рівень вкладеності неможливий без міграції — якщо знадобиться, це окреме рішення. Підзадачі не потрапляють у KPI, у звіти й у лічильники простроченого — свідомо: інакше одна задача з 24 кроками переважила б решту. `checklist*` -процедури внесені в exempt-список `test:audit-coverage` , бо `ActivityLog` веде тільки батьківська задача.

ADR-0006 — Voting freezes the standard document; REJECT clones

- **Дата:** 2026-05-29
- **Контекст:** Голосування довго було "просто рядок" — голос за/проти, статус стандарту змінюється. Документ, який обговорювали, лишався редагованим після закриття → втрачали відповідь на питання «що саме голосували?» і непомітно правили "прийнятий" текст.
- **Рішення:** На закриття голосування активний `STANDARD` -документ замикається (`Document.lockedAt` , `Document.lockedByVotingId` , `Voting.documentId`), у файлове ім'я додається суфікс `(Голосування №N, прийнято|відхилено DD.MM.YYYY)` , `isCurrent` знімається. Сервер (`document.update` / `updateMeta` / `setAsCurrent` / `suggestion.*`) і клієнт (`StandardBodyEditor` через проп `documentLocked`) відмовляють у будь-яких правках — навіть ADMIN. Голосування **ніколи не видаляються** (тільки `vote.adminWipeAll` як emergency escape hatch). При **REJECTED**: `standard.status = DRAFT` (а не REJECTED) + заблокований документ клонується у новий editable з версією `vN+1` — РГ продовжує ітерувати з чистого старту. При **ADOPTED**: статус ADOPTED, новий клон не створюється — заблокований снапшот і є фінал. Поріг голосування — `forVotes / eligibleAtClose > 0.5` (snapshot активних `LEADER/DEPUTY/MEMBER` на момент закриття).
- **Альтернативи:**

- **Видаляти Voting при rejection** — втрачаємо аудит, неможливо пояснити «чому стандарт повернувся в чернетку».
- **Залишати документ editable** — той самий файл потім "правлять", і архівний вердикт стає неспівставним з вмістом.
- **Тримати lock-стан в окремій таблиці** — додатковий join у кожному запиті документів; nullable-FK на Document простіший і вистачає.

- **Наслідки:** `Document.lockedAt != null` ⇒ read-only завжди й для всіх. Унікальність типу (1 STANDARD + 1 TECH_SPEC на стандарт) рахується тільки серед `lockedAt: null` — заблоковані снапшоти стекаються без обмежень. Архівні votings з історичних даних можуть мати `eligibleAtClose = null` → UI має fallback на поточний `eligibleVoters` або `(for+against)`. Усі нові процедури з document-target повинні перевіряти `lockedAt`.

ADR-0005 — Destructive enum changes via boot-time SQL backfill

- **Дата:** 2026-05-29
- **Контекст:** ADR-0004 фіксує `prisma db push --accept-data-loss` на старті — це нормально для nullable-додавання, але деструктивне для enum rename/drop: Prisma drop-and-recreates enum, а існуючі рядки з удаленим значенням блокують операцію. Конкретно: треба було перейменувати `DocumentType.DRAFT_STANDARD` → `STANDARD` і прибрати `FINAL`.
- **Рішення:** Перед `prisma db push` запускати `scripts/pre-db-push.sh` (з `Dockerfile` CMD). Скрипт перевіряє `pg_type` на існування `DocumentType` (skip для fresh DB), потім виконує `scripts/pre-db-push.sql`: `ALTER TYPE "DocumentType" ADD VALUE IF NOT EXISTS 'STANDARD'`; `UPDATE documents SET type = 'STANDARD' WHERE type::text = 'DRAFT_STANDARD'`; `UPDATE documents SET type = 'ATTACHMENT' WHERE type::text = 'FINAL'`. Тільки після цього `prisma db push --accept-data-loss` безпечно дропає старі enum-значення. Скрипт ідемпотентний — `IF NOT EXISTS` + `type::text` comparison + рядки вже мігровані наступного разу.
- **Альтернативи:**
  - **Перейти на `prisma migrate deploy`** — правильне рішення довгостроково, але вимагає створення `prisma/migrations/`-історії з нуля; зараз = post-launch.
  - **Прибрати `--accept-data-loss`** — Prisma відмовиться видалити enum value → деплой падає.
  - **Ручна SQL-міграція через psql на проді** — несумісно з push-to-deploy моделлю.
- **Наслідки:** Кожна наступна деструктивна enum-зміна потребує оновлення `pre-db-push.sql` + перевірки на ідемпотентність. Скрипт виконується **до** Prisma, тож DocumentType enum referenced rawly через `type::text`. Лосс семантики при `FINAL → ATTACHMENT` зафіксований відкрито. Перехід на справжні міграції лишається кандидатом на post-launch (ADR-0004 ризик зростає з кожним подібним кейсом).

ADR-0004 — `db push` замість міграцій Prisma

- **Дата:** 2026-05-26 (зафіксовано ретроспективно)
- **Контекст:** Railway start-команда виконує `prisma db push --accept-data-loss && prisma:seed && start` на кожному деплої. У репо немає каталогу `prisma/migrations/`.

- **Рішення:** схема — source of truth; синхронізація через `db push`. Локально теж `pnpm prisma db push` (НЕ `migrate dev`). Кастомні індекси — окремо в `prisma/indexes.sql`.
- **Альтернативи:** повноцінні міграції (`migrate deploy`) — відкинуто на ранній стадії заради швидкості ітерацій pre-launch.
- **Наслідки:** ⚠️ будь-яка несумісна зміна схеми на проді = `--accept-data-loss` може дропнути колонки/таблиці. Правило: **тільки nullable-додавання** без узгодження; деструктивні зміни — окремо й обережно. Перехід на справжні міграції — кандидат на post-launch.

ADR-0003 — In-process cron (node-cron), без зовнішнього планувальника

- **Дата:** 2026-05-26 (ретроспективно)
- **Контекст:** потрібні нагадування (засідання, дедлайни етапів, закриття голосувань), тижневий дайджест, нічний бекап БД → S3.
- **Рішення:** усе живе в тому самому контейнері через `node-cron`, графіки в `src/instrumentation.ts` (TZ Europe/Kyiv). Ручний тригер: `GET /api/cron/{notifications,digest,backup}` (захист `CRON_SECRET`). Перемикачі `CRON_DISABLED=1`, `CRON_IN_DEV=1`.
- **Альтернативи:** Railway Cron / окремий worker-сервіс — зайва складність для single-instance.
- **Наслідки:** при масштабуванні на кілька instance треба `CRON_DISABLED=1` на всіх, крім одного (інакше дублі). Backend-QA позначив (B-28): `CRON_SECRET` через query-параметр витікає в логи — кандидат прибрати query-fallback, лишити `Authorization: Bearer`.

ADR-0002 — Token-driven theming (CSS-змінні + Tailwind токени)

- **Дата:** 2026-05-26 (ретроспективно)
- **Контекст:** потрібен повний паритет світлої/темної теми на всіх екранах.
- **Рішення:** CSS-змінні `--c-*` у `globals.css`, клас `.dark` на `<html>`, семантичні Tailwind-токени (`bg-card`, `text-ink`, `text-mid`, `text-light`, `bg-page`, `border-hairline`, `bg-pill`, `bg-brand` …). Bootstrap-скрипт у `layout.tsx` ставить клас до гідратації (анти-FOUC).
- **Альтернативи:** `next-themes` — не використано історично; розглядається для усунення F-3 (ThemeProvider state ≠ DOM на першому рендері).
- **Наслідки:** **заборонено** hardcoded `bg-white` / `bg-slate-*` / `text-blue-*` — вони не перемикаються в dark. Lint цього не ловить → ловиться очима в QA (D-1, F-3, F-6). Рішення про спосіб усунення FOUC (cookie vs читання DOM у useState-initializer) — записати окремим ADR при фіксі F-3.

ADR-0001 — RBAC: 3 глобальні ролі × 5 ролей у РГ, DB-override дефолтів

- **Дата:** 2026-05-26 (ретроспективно)

- **Контекст:** держ-домен з тонким розмежуванням прав по робочих групах.
- **Рішення:** глобальні `ADMIN` / `DIRECTOR` / `USER`; у РГ `LEADER` / `DEPUTY` / `SECRETARY` / `MEMBER` / `GUEST`; ~24 actions. Hardcoded `PERMISSIONS`-дефолти перевизначаються таблицею `RolePermission` (кеш у пам'яті, `ensureLoaded()` у tRPC middleware). ADMIN — short-circuit allow; DIRECTOR — повний read + actions зі своєї колонки; `seesAllWorkingGroups()` дає read-усе для ADMIN/DIRECTOR/SECRETARY (per наказ).
- **Альтернативи:** проста role-enum без per-WG — недостатньо для домену.
- **Наслідки:** легко забути перевірку прав на нових процедурах → QA-Backend знайшов цілий клас IDOR/RBAC-bypass (B-1, B-2, B-5, B-6, B-7, B-9). Правило: **кожна нова tRPC-процедура з `byId` / `list` /мутацією має явну `can(...)` або membership-перевірку**; це треба покрити regression-тестами (`tests/security/idor.test.ts`).

ADR-0000 — Прийнято протокол agent-handoff

- **Дата:** 2026-05-27
- **Статус:** Прийнято
- **Контекст:** Агенти змінюються між сесіями (ліміт токенів, фрізи, переключення Kir), губиться контекст, повторюється робота, ламаються раніше прийняті рішення.
- **Рішення:** Прийнято протокол agent-handoff (див. `.claude/skills/agent-handoff/SKILL.md`). Усі агенти зобов'язані читати HANDOFF.md, CONTINUATION.md, останні 15 комітів до початку роботи й оновлювати журнал по віхах. Sync-коміт перед паузою/завершенням сесії — обов'язковий.
- **Альтернативи:** Покладатися на усну передачу контексту через Kir — відкинуто як нестабільне.
- **Наслідки:** Кожна сесія починається з onboarding-чекліста (~5 хв). CONTINUATION.md тримається в актуальному стані. Розгорнуто аналогічно в усіх трьох проєктах Kir (Atommuz, buildco-platform, standardotvoryets).